

Finding Patterns in Arrays

Java Interview Guide — Sri Vasavi to Java Full Stack

Array pattern problems are among the most frequently asked in Java interviews. This guide covers the key techniques with Java code examples and time complexities.

1. Two Pointer Technique

Use two pointers (left and right) moving toward each other or in the same direction. Best for sorted arrays, pair problems, and palindrome checks.

When to use: Pair sum, remove duplicates, container with most water.

```
// Find pair with target sum in sorted array
int left = 0, right = arr.length - 1;
while (left < right) {
    int sum = arr[left] + arr[right];
    if (sum == target) return true;
    else if (sum < target) left++;
    else right--;
} // Time: O(n) | Space: O(1)
```

2. Sliding Window Technique

Maintain a window (subarray) and slide it across the array. Avoids nested loops by reusing previous computations.

When to use: Max/min subarray sum, longest subarray with condition, substrings.

```
// Max sum of subarray of size k
int windowSum = 0, maxSum = 0;
for (int i = 0; i < k; i++) windowSum += arr[i];
maxSum = windowSum;
for (int i = k; i < arr.length; i++) {
    windowSum += arr[i] - arr[i - k];
    maxSum = Math.max(maxSum, windowSum);
} // Time: O(n) | Space: O(1)
```

3. Prefix Sum / Cumulative Sum

Precompute cumulative sums so any range sum query is answered in $O(1)$. Essential for repeated range queries.

When to use: Range sum queries, subarray sum equals k, equilibrium index.

```
// Build prefix sum array
int[] prefix = new int[arr.length + 1];
for (int i = 0; i < arr.length; i++) prefix[i + 1] = prefix[i] + arr[i];
// Sum from index l to r (inclusive)
int rangeSum = prefix[r + 1] - prefix[l];
// Time: O(n) build, O(1) query | Space: O(n)
```

4. HashMap / HashSet for $O(1)$ Lookup

Store elements or their frequencies in a map. Trade space for speed. Very common in pattern detection problems.

When to use: Find duplicate, two sum, frequency count, anagram check.

```
// Two Sum — find indices of two numbers adding to target
Map<Integer, Integer> map = new HashMap<>();
for (int i = 0; i < arr.length; i++) {
    int complement = target - arr[i];
    if (map.containsKey(complement)) return new int[]{map.get(complement), i};
    map.put(arr[i], i);
} // Time: O(n) | Space: O(n)
```

5. Kadane's Algorithm — Maximum Subarray Sum

Track the maximum sum ending at each index. Reset when current sum goes negative. Classic dynamic programming on arrays.

When to use: Maximum subarray, stock buy/sell profit.

```
int maxSoFar = arr[0], maxEndingHere = arr[0]; for (int i = 1; i < arr.length; i++) {
    maxEndingHere = Math.max(arr[i], maxEndingHere + arr[i]); maxSoFar = Math.max(maxSoFar,
    maxEndingHere); } // Time: O(n) | Space: O(1)
```

6. Binary Search on Sorted Arrays

Divide the search space in half each step. Only works on sorted arrays. Used for finding elements, boundaries, and conditions.

When to use: Search element, find first/last occurrence, rotated array search.

```
int left = 0, right = arr.length - 1; while (left <= right) { int mid = left + (right -
left) / 2; if (arr[mid] == target) return mid; else if (arr[mid] < target) left = mid + 1;
else right = mid - 1; } // Time: O(log n) | Space: O(1)
```

7. Sorting-Based Patterns

Sort first, then apply two-pointer or greedy logic. Sorting costs $O(n \log n)$ but simplifies many problems.

When to use: 3-sum, merge intervals, meeting rooms, group anagrams.

```
// Merge overlapping intervals Arrays.sort(intervals, (a, b) -> a[0] - b[0]); List<int[]>
merged = new ArrayList<>(); for (int[] interval : intervals) { if (merged.isEmpty() ||
merged.get(merged.size()-1)[1] < interval[0]) merged.add(interval); else
merged.get(merged.size()-1)[1] = Math.max(merged.get(merged.size()-1)[1], interval[1]);
} // Time: O(n log n) | Space: O(n)
```

Quick Reference — Pattern vs Use Case

Pattern	Best For	Time	Space
Two Pointer	Sorted arrays, pair problems	$O(n)$	$O(1)$
Sliding Window	Subarray/substring conditions	$O(n)$	$O(1)$
Prefix Sum	Range sum queries	$O(n)$	$O(n)$
HashMap/HashSet	Lookup, frequency, duplicates	$O(n)$	$O(n)$
Kadane's	Max subarray sum	$O(n)$	$O(1)$
Binary Search	Sorted array search	$O(\log n)$	$O(1)$
Sorting-based	Intervals, grouping	$O(n \log n)$	$O(n)$

Interview Tip: Always state your pattern choice before coding. Say — 'I will use a sliding window here because this asks for a subarray with a condition.' This shows clear thought process to the interviewer.